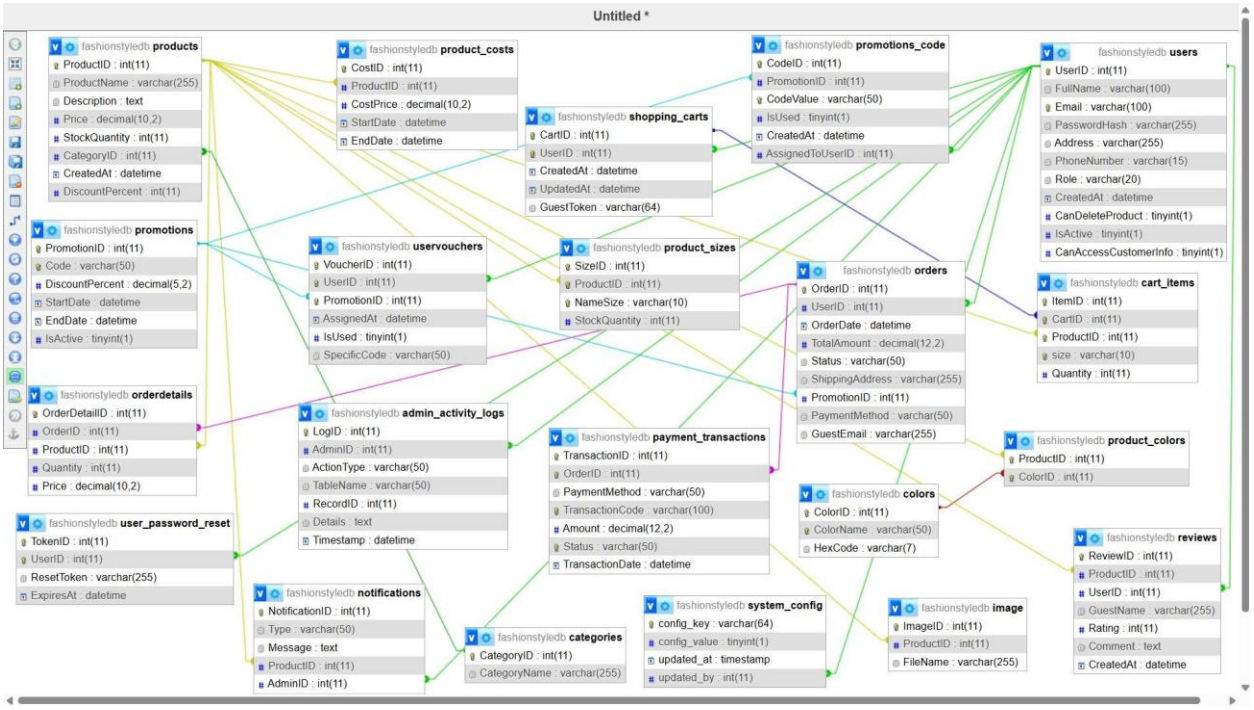
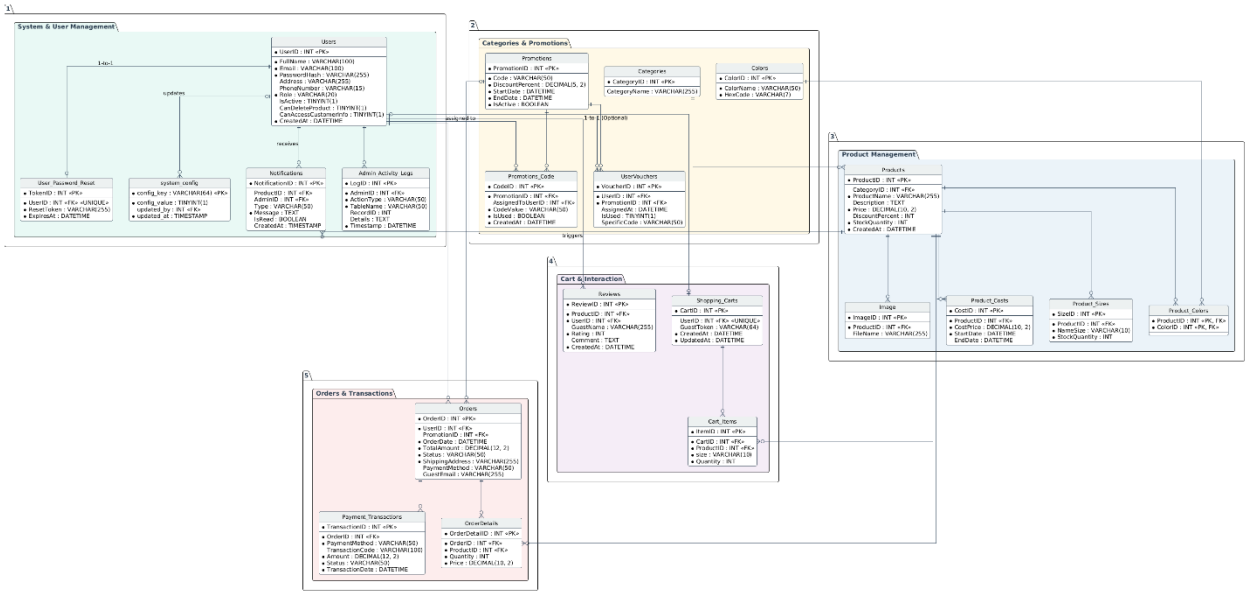
Cơ Sở Dữ liệu và ERD:





ARCHITECTURAL ANALYSIS OF THE FASHIONSTYLE ONLINE DATABASE

To build the FashionStyle Online e-commerce platform, I designed a relational database schema (RDBMS) comprising 21 tables. My core objective was not merely data storage, but ensuring strict Data Integrity, optimizing query performance for RESTful APIs, and guaranteeing absolute synchronization between the Website and Mobile App platforms.

I partitioned the database into 5 logical modules with tightly bound relationships as follows:

1. System & User Management Module

This module serves as the security and operational foundation of the entire system.

- The Users Table (Core Entity): Instead of relying on a single static string field for roles, I designed Boolean flags (IsActive, CanDeleteProduct, CanAccessCustomerInfo) to implement Fine-grained Access Control. This allows me to easily lock violating accounts or flexibly grant specific permissions to administrators without breaking the core role hierarchy.
- Security and Auditing: I applied a closed 1-to-1 relationship between Users and User_Password_Reset via a UNIQUE constraint on UserID. Password reset token issuance is managed separately with a specific lifecycle (ExpiresAt). Simultaneously, the Admin_Activity_Logs table acts as a black box, recording all CRUD operations performed by administrators (AdminID), ensuring a highly transparent Audit Trail.
- Dynamic Configuration: The system_config table allows me to store operational settings (such as enabling maintenance mode) as Key-Value pairs without hardcoding them into the source code. The updated_by foreign key pointing back to Users helps me track exactly who last modified the system configuration.

2. Product Management & Variants Module

The most complex aspect of the fashion industry is managing product variants like colors and sizes. I solved this challenge through high-level data normalization:

- Solving the Color Variant Problem (M-to-N Relationship): A product can have multiple colors, and a single color can belong to multiple products. Therefore, I did not store colors

directly in the Products table. Instead, I separated them into a Colors table (storing the HexCode for Frontend rendering). Then, I utilized a junction table named Product_Colors containing a Composite Key (ProductID and ColorID). This design drastically optimizes query speed when filtering products by specific color palettes.

- Size and Granular Inventory Management (1-to-N Relationship): I designed the Product_Sizes table to store size labels (S, M, L). The crucial architectural decision here was placing the StockQuantity field in this table rather than the Products table. As a result, my system can accurately track inventory for each specific variant (e.g., identifying that a shirt is out of size M but still available in size L). I also enforced a UNIQUE KEY (ProductID, NameSize) constraint to prevent duplicate size entries that could cause logical inventory errors.
- Historical Cost Management: The Product_Costs table was created to store the history of cost price changes over time (StartDate, EndDate). By decoupling this from the Products table, I can generate accurate gross profit margin reports for any past period, unaffected by current fluctuations in import costs.

3. Categories & Promotions Module

- I designed a multi-tier promotion mechanism. The Promotions table manages overall, public discount programs. However, to support highly personalized Marketing campaigns, I implemented the UserVouchers table. This acts as an M-to-N bridge between Users and Promotions, allowing the system to dynamically "push" customized discount codes (SpecificCode) directly into a specific customer's private voucher wallet.

4. Cart & Guest Interaction Module

To maximize the Conversion Rate, I designed a shopping cart system that natively supports both registered members and Guest users.

- The Shopping_Carts Table: I proactively allowed the UserID foreign key to accept NULL values and introduced a GuestToken field. When a guest user adds an item to their cart, the system issues a device-specific identification token. If they decide to register an account

later during checkout, I simply update the UserID onto this existing cart, guaranteeing a frictionless and seamless shopping experience.

- Cart Details (Cart_Items): I utilized a UNIQUE KEY (CartID, ProductID, size) constraint. This ensures that when a user clicks "Add to cart" for a product and size they have already selected, the system will not generate a redundant new row. Instead, it natively understands to execute an UPDATE command to increment the Quantity.
- Product Reviews (Reviews): Similarly, I added the GuestName field to expand the review functionality to non-registered users, which significantly boosts user-generated content and engagement for the products.

5. Orders & Transactions Module

Financial data integrity is the absolute top priority in my architecture.

- Order Immutability: In the OrderDetails table, I designed an independent Price field. Although the current product price exists in the Products table, capturing and copying the price to OrderDetails at the exact moment of checkout "freezes" the transaction value. If I update the product's selling price in the future, the historical revenue data of past orders remains strictly unaltered.
- Independent Payment Flow: The Payment_Transactions table is deliberately separated from Orders to independently manage the Payment Gateway flow. A single order can have multiple payment attempts (Transactions). This decoupling allows me to cleanly reconcile the TransactionCode and smoothly handle edge cases where a customer experiences a payment failure and attempts to pay again later.

```
-- ==================================================================
-- SCRIPT DATABASE FASHIONSTYLE ONLINE (TỐI ƯU HÓA - ĐỦ 21 BẢNG)
-- Trình tự chuẩn: Bảng Độc Lập -> Bảng Trung Tâm -> Bảng Chi Tiết -> Bảng Nghiệp Vụ
-- ==================================================================

SET FOREIGN_KEY_CHECKS = 0;

-- ==========================================================
-- NHÓM 1: CÁC BẢNG ĐỘC LẬP (Không phụ thuộc khóa ngoại)
-- ==========================================================

-- 1. Bảng lưu trữ người dùng và quyền hạn
CREATE TABLE IF NOT EXISTS Users (
    UserID INT PRIMARY KEY AUTO_INCREMENT,
    FullName VARCHAR(100) NOT NULL,
    Email VARCHAR(100) NOT NULL UNIQUE,
    PasswordHash VARCHAR(255) NOT NULL,
    Address VARCHAR(255) NULL,
    PhoneNumber VARCHAR(15) NULL,
    Role VARCHAR(20) NOT NULL DEFAULT 'Customer',
    IsActive TINYINT(1) DEFAULT 1,
    CanDeleteProduct TINYINT(1) DEFAULT 0,
    CanAccessCustomerInfo TINYINT(1) DEFAULT 1,
    CreatedAt DATETIME DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;
```

```
-- 2. Bảng danh mục sản phẩm
CREATE TABLE IF NOT EXISTS Categories (
    CategoryID INT PRIMARY KEY AUTO_INCREMENT,
    CategoryName VARCHAR(255) NULL
) ENGINE=InnoDB;

-- 3. Bảng danh mục màu sắc
CREATE TABLE IF NOT EXISTS Colors (
    ColorID INT PRIMARY KEY AUTO_INCREMENT,
    ColorName VARCHAR(50) NOT NULL UNIQUE,
    HexCode VARCHAR(7) NOT NULL
) ENGINE=InnoDB;

-- 4. Bảng chương trình khuyến mãi tổng quát
CREATE TABLE IF NOT EXISTS Promotions (
    PromotionID INT PRIMARY KEY AUTO_INCREMENT,
    Code VARCHAR(50) NOT NULL UNIQUE,
    DiscountPercent DECIMAL(5, 2) NOT NULL,
    StartDate DATETIME NOT NULL,
    EndDate DATETIME NOT NULL,
    IsActive BOOLEAN NOT NULL DEFAULT 1
) ENGINE=InnoDB;

-- ==========================================================
-- NHÓM 2: CÁC BẢNG CỐT LÕI & HỆ THỐNG
```

```
-- ==========================================================

-- 5. Bảng cấu hình hệ thống
CREATE TABLE IF NOT EXISTS system_config (
    config_key VARCHAR(64) NOT NULL,
    config_value TINYINT(1) NOT NULL DEFAULT 0,
    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    updated_by INT(11) NULL DEFAULT NULL COMMENT 'Admin UserID người cập nhật',
    CONSTRAINT pk_system_config PRIMARY KEY (config_key),
    CONSTRAINT fk_sysconfig_user FOREIGN KEY (updated_by) REFERENCES Users(UserID) ON DELETE SET NULL ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- 6. Bảng sản phẩm chính
CREATE TABLE IF NOT EXISTS Products (
    ProductID INT PRIMARY KEY AUTO_INCREMENT,
    ProductName VARCHAR(255) NOT NULL,
    Description TEXT NULL,
    Price DECIMAL(10, 2) NOT NULL,
    DiscountPercent INT DEFAULT 0,
    StockQuantity INT NOT NULL DEFAULT 0,
    CategoryID INT NULL,
    CreatedAt DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (CategoryID) REFERENCES Categories(CategoryID)
) AUTO_INCREMENT = 101, ENGINE=InnoDB;
```

```
-- ==========================================================
-- NHÓM 3: CÁC BẢNG CHI TIẾT SẢN PHẨM & BIẾN THỂ
-- ==========================================================

-- 7. Bảng hình ảnh sản phẩm
CREATE TABLE IF NOT EXISTS Image (
    ImageID INT PRIMARY KEY AUTO_INCREMENT,
    ProductID INT NOT NULL,
    FileName VARCHAR(255) NULL,
    FOREIGN KEY (ProductID) REFERENCES Products(ProductID) ON DELETE CASCADE
) AUTO_INCREMENT = 101, ENGINE=InnoDB;

-- 8. Bảng kích cỡ và tồn kho chi tiết
CREATE TABLE IF NOT EXISTS Product_Sizes (
    SizeID INT PRIMARY KEY AUTO_INCREMENT,
    ProductID INT NOT NULL,
    NameSize VARCHAR(10) NOT NULL,
    StockQuantity INT NOT NULL DEFAULT 0,
    FOREIGN KEY (ProductID) REFERENCES Products(ProductID) ON DELETE CASCADE,
    UNIQUE KEY UQ_Product_Size (ProductID, NameSize)
) ENGINE=InnoDB;

-- 9. Bảng trung gian Sản phẩm & Màu sắc
CREATE TABLE IF NOT EXISTS Product_Colors (
    ProductID INT NOT NULL,
    ColorID INT NOT NULL,
```

```
    PRIMARY KEY (ProductID, ColorID),
    CONSTRAINT fk_prodcolor_product FOREIGN KEY (ProductID) REFERENCES Products(ProductID) ON DELETE CASCADE,
    CONSTRAINT fk_prodcolor_color FOREIGN KEY (ColorID) REFERENCES Colors(ColorID) ON DELETE CASCADE
) ENGINE=InnoDB;

-- 10. Bảng giá vốn
CREATE TABLE IF NOT EXISTS Product_Costs (
    CostID INT PRIMARY KEY AUTO_INCREMENT,
    ProductID INT NOT NULL,
    CostPrice DECIMAL(10, 2) NOT NULL CHECK (CostPrice >= 0),
    StartDate DATETIME NOT NULL,
    EndDate DATETIME NULL,
    FOREIGN KEY (ProductID) REFERENCES Products(ProductID) ON DELETE CASCADE
) ENGINE=InnoDB;

-- ==========================================================
-- NHÓM 4: KHUYẾN MÃI, TƯƠNG TÁC & NHẬT KÝ
-- ==========================================================

-- 11. Bảng mã giảm giá chi tiết
CREATE TABLE IF NOT EXISTS Promotions_Code (
    CodeID INT PRIMARY KEY AUTO_INCREMENT,
    PromotionID INT NOT NULL,
    AssignedToUserID INT NULL,
    CodeValue VARCHAR(50) NOT NULL UNIQUE,
```

```
    IsUsed BOOLEAN NOT NULL DEFAULT 0,
    CreatedAt DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (PromotionID) REFERENCES Promotions(PromotionID) ON DELETE CASCADE,
    FOREIGN KEY (AssignedToUserID) REFERENCES Users(UserID) ON DELETE SET NULL
) ENGINE=InnoDB;

-- 12. Bảng Voucher người dùng cá nhân
CREATE TABLE IF NOT EXISTS UserVouchers (
    VoucherID INT AUTO_INCREMENT PRIMARY KEY,
    UserID INT NOT NULL,
    PromotionID INT NOT NULL,
    AssignedAt DATETIME DEFAULT CURRENT_TIMESTAMP,
    IsUsed TINYINT(1) DEFAULT 0,
    SpecificCode VARCHAR(50) NULL,
    FOREIGN KEY (UserID) REFERENCES Users(UserID) ON DELETE CASCADE,
    FOREIGN KEY (PromotionID) REFERENCES Promotions(PromotionID) ON DELETE CASCADE,
    UNIQUE KEY uq_user_promo (UserID, PromotionID)
) ENGINE=InnoDB;

-- 13. Đánh giá từ khách hàng (Hỗ trợ Guest)
CREATE TABLE IF NOT EXISTS Reviews (
    ReviewID INT PRIMARY KEY AUTO_INCREMENT,
    ProductID INT NOT NULL,
    UserID INT NULL,
    GuestName VARCHAR(255) NULL,
    Rating INT NOT NULL CHECK (Rating >= 1 AND Rating <= 5),
```

```
    Comment TEXT NULL,
    CreatedAt DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (ProductID) REFERENCES Products(ProductID) ON DELETE CASCADE,
    FOREIGN KEY (UserID) REFERENCES Users(UserID) ON DELETE SET NULL
) AUTO_INCREMENT = 401, ENGINE=InnoDB;

-- 14. Cấp lại mật khẩu
CREATE TABLE IF NOT EXISTS User_Password_Reset (
    TokenID INT PRIMARY KEY AUTO_INCREMENT,
    UserID INT NOT NULL UNIQUE,
    ResetToken VARCHAR(255) NOT NULL,
    ExpiresAt DATETIME NOT NULL,
    FOREIGN KEY (UserID) REFERENCES Users(UserID) ON DELETE CASCADE
) ENGINE=InnoDB;

-- 15. Nhật ký hành động Admin
CREATE TABLE IF NOT EXISTS Admin_Activity_Logs (
    LogID INT PRIMARY KEY AUTO_INCREMENT,
    AdminID INT NOT NULL,
    ActionType VARCHAR(50) NOT NULL,
    TableName VARCHAR(50) NOT NULL,
    RecordID INT NULL,
    Details TEXT NULL,
    Timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (AdminID) REFERENCES Users(UserID) ON DELETE CASCADE
) ENGINE=InnoDB;
```

```
-- 16. Thông báo hệ thống
CREATE TABLE IF NOT EXISTS Notifications (
    NotificationID INT AUTO_INCREMENT PRIMARY KEY,
    Type VARCHAR(50) DEFAULT 'SYSTEM',
    Message TEXT NOT NULL,
    ProductID INT NULL,
    AdminID INT NULL,
    IsRead BOOLEAN DEFAULT FALSE,
    CreatedAt DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (ProductID) REFERENCES Products(ProductID) ON DELETE CASCADE,
    FOREIGN KEY (AdminID) REFERENCES Users(UserID) ON DELETE SET NULL
) ENGINE=InnoDB;

-- ==========================================================
-- NHÓM 5: GIỎ HÀNG VÀ ĐƠN HÀNG (NGHIỆP VỤ MUA BÁN)
-- ==========================================================

-- 17. Giỏ hàng của người dùng (Hỗ trợ Guest)
CREATE TABLE IF NOT EXISTS Shopping_Carts (
    CartID INT PRIMARY KEY AUTO_INCREMENT,
    UserID INT NULL UNIQUE,
    GuestToken VARCHAR(64) NULL DEFAULT NULL,
    CreatedAt DATETIME DEFAULT CURRENT_TIMESTAMP,
    UpdatedAt DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (UserID) REFERENCES Users(UserID) ON DELETE CASCADE
```

```
) ENGINE=InnoDB;

-- 18. Các món đồ trong giỏ hàng
CREATE TABLE IF NOT EXISTS Cart_Items (
    ItemID INT PRIMARY KEY AUTO_INCREMENT,
    CartID INT NOT NULL,
    ProductID INT NOT NULL,
    size VARCHAR(10) NOT NULL,
    Quantity INT NOT NULL CHECK (Quantity > 0),
    FOREIGN KEY (CartID) REFERENCES Shopping_Carts(CartID) ON DELETE CASCADE,
    FOREIGN KEY (ProductID) REFERENCES Products(ProductID) ON DELETE CASCADE,
    UNIQUE KEY UQ_Cart_Product_Size (CartID, ProductID, size)
) ENGINE=InnoDB;

-- 19. Thông tin đơn hàng tổng quát
CREATE TABLE IF NOT EXISTS Orders (
    OrderID INT PRIMARY KEY AUTO_INCREMENT,
    UserID INT NULL,
    GuestEmail VARCHAR(255) NULL,
    OrderDate DATETIME DEFAULT CURRENT_TIMESTAMP,
    TotalAmount DECIMAL(12, 2) NOT NULL,
    Status VARCHAR(50) NOT NULL DEFAULT 'Chờ xử lý',
    ShippingAddress VARCHAR(255) NOT NULL,
    PaymentMethod VARCHAR(50) NULL,
    PromotionID INT NULL,
    FOREIGN KEY (UserID) REFERENCES Users(UserID) ON DELETE SET NULL,
```

```
    FOREIGN KEY (PromotionID) REFERENCES Promotions(PromotionID) ON DELETE SET NULL
) AUTO_INCREMENT = 201, ENGINE=InnoDB;

-- 20. Chi tiết từng sản phẩm trong đơn hàng
CREATE TABLE IF NOT EXISTS OrderDetails (
    OrderDetailID INT PRIMARY KEY AUTO_INCREMENT,
    OrderID INT NOT NULL,
    ProductID INT NOT NULL,
    Quantity INT NOT NULL,
    Price DECIMAL(10, 2) NOT NULL,
    FOREIGN KEY (OrderID) REFERENCES Orders(OrderID) ON DELETE CASCADE,
    FOREIGN KEY (ProductID) REFERENCES Products(ProductID) ON DELETE CASCADE
) AUTO_INCREMENT = 301, ENGINE=InnoDB;

-- 21. Lịch sử giao dịch thanh toán
CREATE TABLE IF NOT EXISTS Payment_Transactions (
    TransactionID INT PRIMARY KEY AUTO_INCREMENT,
    OrderID INT NOT NULL,
    PaymentMethod VARCHAR(50) NOT NULL,
    TransactionCode VARCHAR(100) UNIQUE NULL,
    Amount DECIMAL(12, 2) NOT NULL CHECK (Amount > 0),
    Status VARCHAR(50) NOT NULL CHECK (Status IN ('Thành công', 'Thất bại', 'Đang chờ')),
    TransactionDate DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (OrderID) REFERENCES Orders(OrderID) ON DELETE CASCADE,
    UNIQUE KEY UQ_Order_Success (OrderID, Status)
) ENGINE=InnoDB;
```

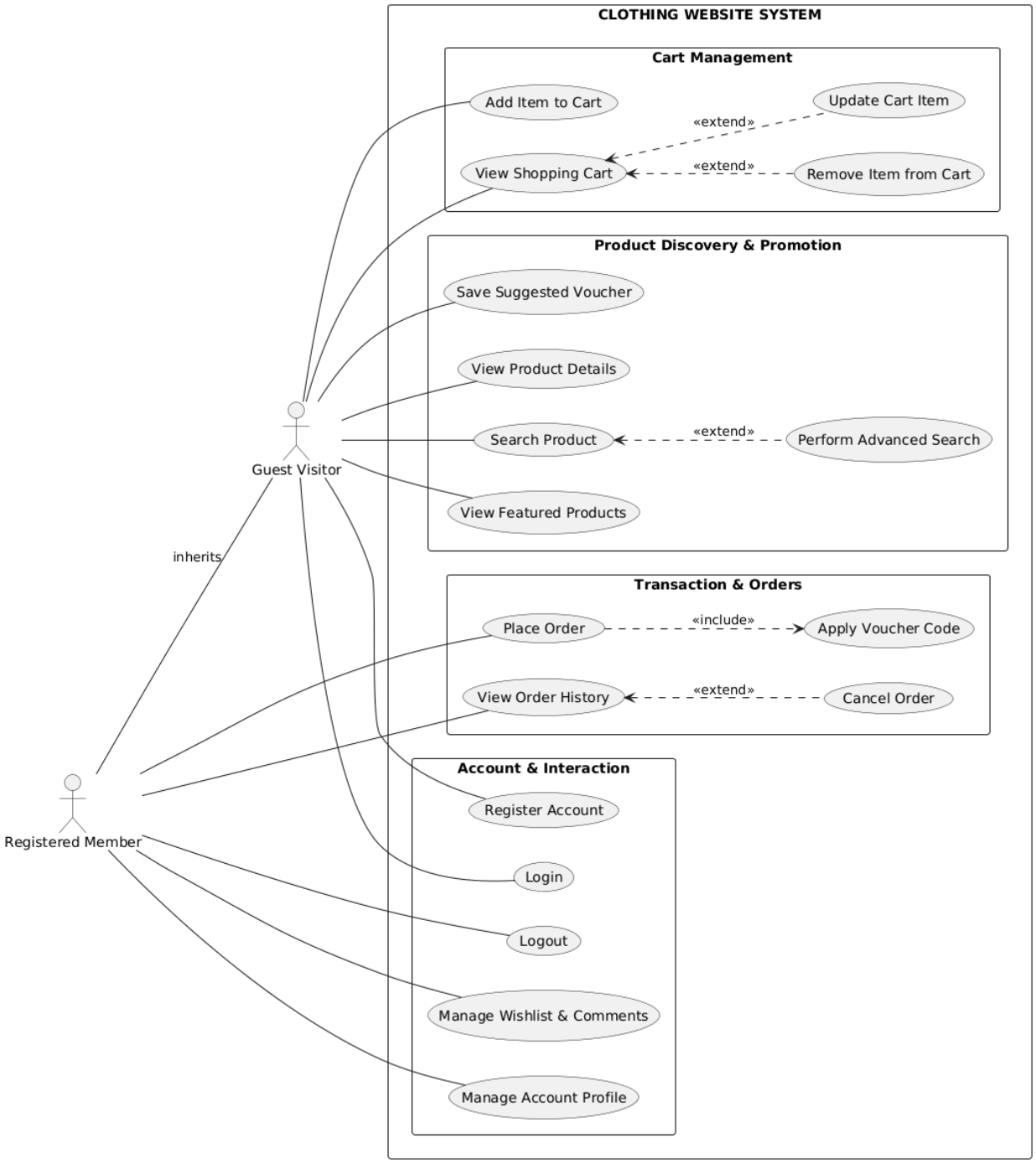
```
-- ====================================================================
-- SCRIPT NẠP DỮ LIỆU MẪU (MOCK DATA)
-- ====================================================================

-- Reset lại các chỉ số ID tự tăng về ban đầu (Phòng trường hợp chạy lại)
ALTER TABLE Categories AUTO_INCREMENT = 1;
ALTER TABLE Colors AUTO_INCREMENT = 1;
ALTER TABLE Products AUTO_INCREMENT = 101;
ALTER TABLE Image AUTO_INCREMENT = 101;

-- Nạp cấu hình hệ thống
INSERT IGNORE INTO system_config (config_key, config_value) VALUES
 ('maintenance_mode',   0),
 ('telegram_alerts',    1),
 ('close_registration', 0);

-- Nạp danh mục
INSERT IGNORE INTO Categories (CategoryID, CategoryName) VALUES
(1, 'Special Edition'),
(2, 'Luxury Collection'),
(3, 'Summer Edition'),
(4, 'Unique Collection');

-- Nạp màu sắc
```

```
INSERT IGNORE INTO Colors (ColorID, ColorName, HexCode) VALUES

(1, 'BLACK', '#000000'),

(2, 'RED', '#FF0000'),

(3, 'BLUE', '#0000FF'),

(4, 'WHITE', '#FFFFFF'),

(5, 'ROSE', '#FF007F'),

(6, 'GREEN', '#00FF00');


-- Nạp sản phẩm

INSERT IGNORE INTO Products (ProductID, ProductName, Price, StockQuantity, CategoryID,
Description) VALUES

(101, 'Luxury Dress', 3500.00, 10, 1, 'Luxury Dress Special Edition'),

(102, 'Luxury Black Clothing', 1050.00, 20, 2, 'Luxury Black Clothing Collection'),

(103, 'Luxury Blue Dress', 5000.00, 5, 3, 'Luxury Blue Dress Summer Edition'),

(104, 'Special Brown Dress', 2500.00, 15, 4, 'Special Brown Dress Unique Collection'),

(105, 'Special Luxury Dress', 5200.00, 0, 4, 'Special Luxury Dress Unique Collection'),

(106, 'Super Luxury Dress', 8500.00, 20, 2, 'Super Luxury Dress Luxury Collection'),

(107, 'Luxury Brown Dress', 4500.00, 500, 2, 'Luxury Brown Dress Luxury Collection'),

(108, 'Premium Brown Dress', 4500.00, 500, 4, 'Premium Brown Dress Unique Collection'),

(109, 'Luxury White Dress', 4500.00, 500, 2, 'Luxury White Dress Luxury Collection'),

(110, 'Brown Limited Collection', 4500.00, 500, 4, 'Brown Limited Collection Unique'),

(111, 'Golden Premium T-shirt', 4500.00, 500, 2, 'Golden Premium T-shirt Luxury'),

(112, 'Golden T-shirt', 4500.00, 500, 3, 'Golden T-shirt Summer Edition'),

(113, 'White Premium T-shirt', 4500.00, 500, 1, 'White Premium T-shirt Special'),

(114, 'Brown Premium T-shirt', 4500.00, 500, 1, 'Brown Premium T-shirt Special'),

(115, 'Brown Special Clothing', 4500.00, 500, 1, 'Brown Special Clothing Special'),
```

```
(116, 'White Special Clothing', 4500.00, 500, 1, 'White Special Clothing Special'),
(117, 'Dark Special Clothing', 4500.00, 500, 1, 'Dark Special Clothing Special'),
(118, 'Brown Premium Clothing', 4500.00, 500, 4, 'Brown Premium Clothing Unique');

-- Nạp Sizes cho sản phẩm
INSERT IGNORE INTO Product_Sizes (ProductID, NameSize, StockQuantity)
SELECT p.ProductID, s.SizeName, 100
FROM Products p
CROSS JOIN (
    SELECT 'XS' AS SizeName UNION ALL
    SELECT 'SM' UNION ALL
    SELECT 'M' UNION ALL
    SELECT 'LG' UNION ALL
    SELECT 'XL' UNION ALL
    SELECT '2XL'
) s;

-- Liên kết màu sắc
INSERT IGNORE INTO Product_Colors (ProductID, ColorID)
SELECT ProductID, 1 FROM Products WHERE ProductID % 2 = 0
UNION ALL
SELECT ProductID, 4 FROM Products WHERE ProductID % 2 <> 0;

SET FOREIGN_KEY_CHECKS = 1;
```

3. Mô tả chi tiết các chức năng hệ thống

A. Phân hệ dành cho Khách hàng (Frontend)



PART 1: ACCOUNT & INTERACTION GROUP

UC-WEB-01: Register Account

- Summary: As a Guest Visitor, I want to create an account to become a member.
- Actor: Guest Visitor.
- Trigger: User clicks "Register".
- Pre-condition: None.
- Basic Flow: 1. User enters info -> 2. System validates -> 3. System saves to DB -> 4. Notify success.
- Exception Flow: Email already exists -> System notifies error.

UC-WEB-02: Login

- Summary: As a Guest Visitor, I want to log in to access member-only features.
- Actor: Guest Visitor.
- Trigger: User clicks "Login".
- Pre-condition: Account exists.
- Basic Flow: 1. User enters Email/Password -> 2. System validates JWT -> 3. Grant access.
- Business Rule: Passwords must be hashed via bcryptjs.

UC-WEB-03: Logout

- Summary: As a Member, I want to end my session securely.
- Actor: Registered Member.
- Basic Flow: 1. User clicks "Logout" -> 2. System clears JWT/Session -> 3. Redirect to Home.

UC-WEB-04: Manage Account Profile

- Summary: As a Member, I want to update my personal info and shipping address.
- Actor: Registered Member.
- Post-condition: Profile updated in DB.

UC-WEB-05: Manage Wishlist & Comments

- Summary: As a Member, I want to save favorite products and write reviews.
- Actor: Registered Member.

PART 2: PRODUCT DISCOVERY & PROMOTION GROUP

UC-WEB-06: Search Product & Advanced Search

- Summary: As a Guest/Member, I want to find products by keywords or filters.
- Actor: Guest Visitor (Inherited by Member).
- Basic Flow: 1. Input keyword -> 2. System filters DB -> 3. Show results.
- Alternative Flow (Advanced): Filter by Color (Red, Black...), Category, or Price range.

UC-WEB-07: View Featured Products & Details

- Summary: As a User, I want to see homepage collections and full product info.
- Actor: Guest Visitor.

UC-WEB-08: Save Suggested Voucher

- Summary: As a User, I want to save promotional codes for later use.
- Actor: Guest Visitor.

PART 3: CART MANAGEMENT TEAM

UC-WEB-09: Add Item to Cart

- Summary: As a User, I want to add products with specific size/color to my cart.
- Actor: Guest Visitor.
- Post-condition: Item added to cartSlice (Redux) and DB.

UC-WEB-10: View & Update/Remove Cart Item

- Summary: As a User, I want to review, change quantity, or delete items in my cart.
- Actor: Guest Visitor.

PART 4: TRANSACTION & ORDERS GROUP

UC-WEB-11: Place Order (Include: Apply Voucher)

- Summary: As a Member, I want to pay for my cart items.
- Actor: Registered Member.
- Trigger: Clicks "Place Order".
- Basic Flow: 1. Verify Cart -> 2. Apply Voucher (Include) -> 3. Save Order via API -> 4. Send Email via Nodemailer.

UC-WEB-12: View Order History & Cancel Order

- Summary: As a Member, I want to track my orders or cancel pending ones.
- Actor: Registered Member.
- Exception Flow: Cannot cancel if order status is "Shipped".

DETAILED SPECIFICATION TABLE (EXAMPLE STANDARD THINKING NOTES FOR USE CASE "SEARCH PRODUCT")

USE CASE SPECIFICATION

Use Case ID	UC-WEB-06
Use Case Name	Search Product
Description	As a Guest Visitor, I want to find products by keyword to quickly locate the items I need.
Actor(s)	Guest Visitor, Registered Member (Inherits)
Priority	Must Have
Trigger	User enters text in the search bar and presses Enter or clicks the Search icon.
Pre-Condition	System is online and connected to the Product Database.
Post-Condition	System displays a list of matching products or a "Not Found" message.
Basic Flow	1. User accesses the Website. 2. User enters keywords in the search bar. 3. System calls /api/products with search parameters. 4. System displays matching products on the Search Result page.
Alternative Flow	2a. Perform Advanced Search (Extend): 2a1. User clicks "Filter". 2a2. User selects Color, Category, or Price range. 2a3. System returns refined results.

Exception Flow	4a. No results found: System displays "No products match your search" and suggests featured items.
Business Rules	BR-SEARCH: Keywords must be at least 2 characters long.
Non-Functional	NFR-SEARCH: Search results must load within 2 seconds.

Group 1: Account & Interaction

- Register Account: Allows guest visitors to create a new account to become a registered member.
- Login: Allows guest visitors to authenticate their credentials (entering email and password) to access the system and be officially recognized as a registered member.
- Logout: Allows logged-in members to safely end and exit their current session, returning them to the guest visitor state.
- Manage Wishlist & Comments: Members can add products to their wishlist for later viewing and write review comments about the products.
- Manage Account Profile: Members can view and edit their personal information, shipping addresses, and change their password.

Group 2: Product Discovery & Promotion

- View Featured Products: Displays new, best-selling, or promotional products right on the homepage to attract users.
- Save Suggested Voucher: Users can save discount codes suggested by the system (e.g., from advertising banners) to their account for later use.
- Perform Advanced Search: Allows users to search for products based on various specific criteria (such as name, category, price range, color, and size).

- View Product Details: Displays comprehensive information about a specific product, including images, descriptions, price, available sizes and colors, and customer reviews.

Group 3: Cart Management

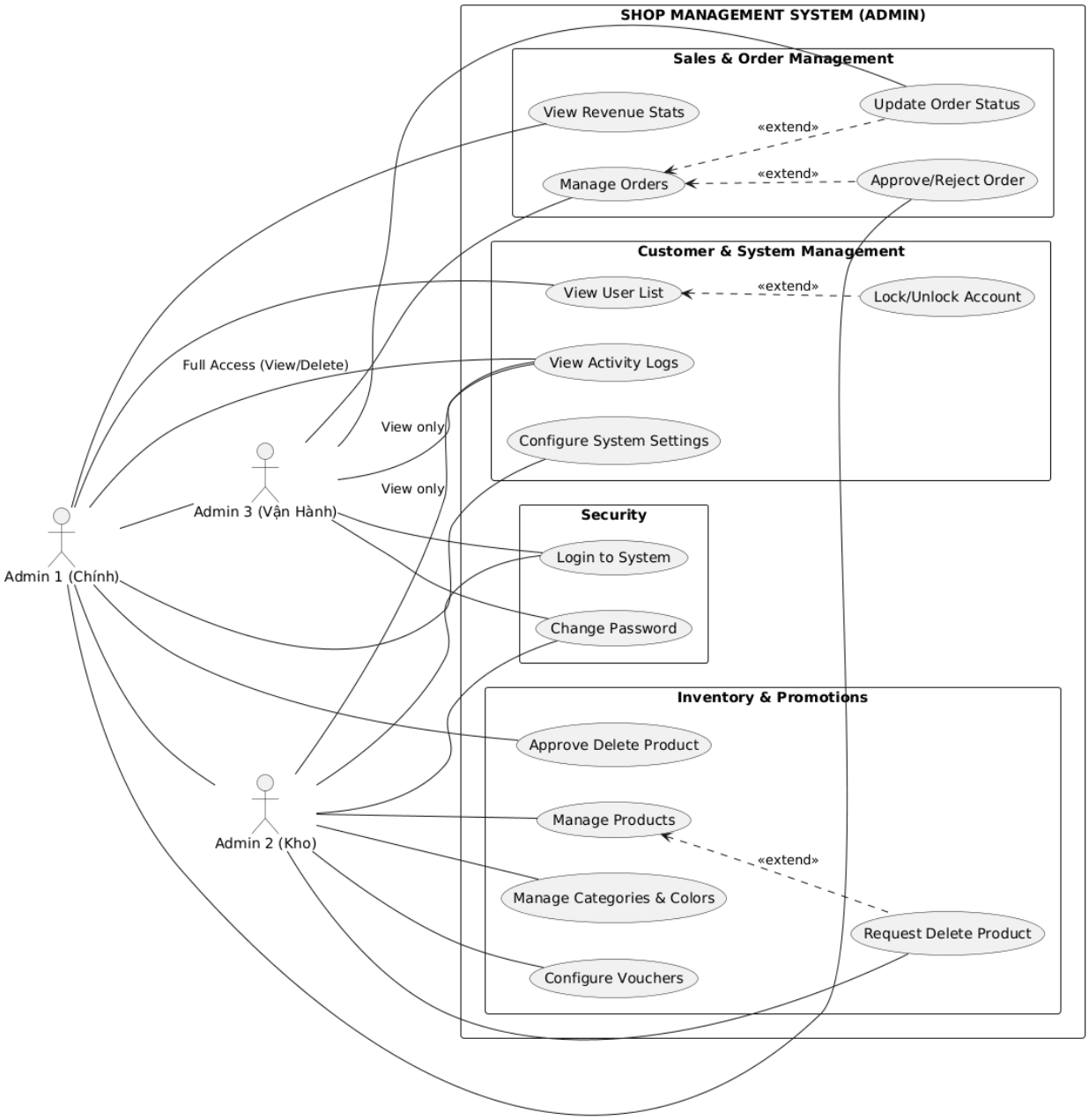
- Add Item to Cart: Users select a product (including its specific size and color) and add it to their virtual shopping cart.
- View Shopping Cart: Displays the list of selected products and the estimated total amount.
- Update Cart Item: An extension function when viewing the cart, allowing users to modify the quantity of the selected products.
- Remove Item from Cart: An extension function when viewing the cart, allowing users to remove a product they no longer wish to purchase.

Group 4: Transaction & Orders

- Place Order: The checkout process for the products currently in the cart. This includes entering the shipping address and selecting a payment method.
- Apply Voucher Code: An included step in the checkout process where the system calculates and deducts the discount amount if the entered voucher code is valid.
- View Order History: Members can review the list of their previously placed orders along with their current statuses.
- Cancel Order: An extension function from viewing the order history, allowing members to cancel orders that are still in the "pending confirmation" (unprocessed) state.
- Perform Advanced Search: Allows users to filter products more thoroughly based on specific criteria such as price range, color, size, and specific categories.

B. Phân hệ Quản trị (Admin)

Dành cho chủ cửa hàng để điều hành toàn bộ hoạt động kinh doanh.



GROUP 1: SALES & ORDER MANAGEMENT

UC-ADM-01: Manage Orders (Include: Update & Approve/Reject)

- **Summary:** As an **Admin**, I want to manage customer orders to ensure correct fulfillment.
- **Actors:** Admin 3 (Operator), Admin 1 (Main Admin).
- **Basic Flow:** 1. Admin views order list -> 2. Admin selects an order -> 3. Admin updates status or approves.
- **Alternative Flow (Approve/Reject):** Admin 3 proposes -> Admin 1 Reviews -> Approve/Reject (Ref: Order flow in architecture).

UC-ADM-02: View Revenue Stats

- **Summary:** As **Admin 1**, I want to view revenue charts to track shop performance.
- **Actor:** Admin 1 (Main Admin).
- **Business Rule:** Only Admin Level 1 (admin1@fashionstyle.com) has access to this sensitive financial data.

GROUP 2: CUSTOMER & SYSTEM MANAGEMENT

UC-ADM-03: View User List (Extend: Lock/Unlock Account)

- **Summary:** As **Admin 1**, I want to manage customer accounts to maintain system security.
- **Actor:** Admin 1 (Main Admin).
- **Business Rule:** Admin 1 can toggle the IsActive field in the database to lock suspicious accounts.

UC-ADM-04: View Activity Logs

- **Summary:** As an **Admin**, I want to see the history of actions to monitor system changes.
- **Actors:** Admin 1 (Full Access), Admin 2 & 3 (View Only).

- **Technical Detail:** Records all CRUD actions into the admin_logs table (Ref: admin_logs.js API).

UC-ADM-05: Configure System Settings

- **Summary:** As **Admin 1**, I want to adjust global system parameters.
- **Actor:** Admin 1.

GROUP 3: INVENTORY & PROMOTIONS

UC-ADM-06: Manage Products (Extend: Request/Approve Delete)

- **Summary:** As an **Admin**, I want to add or edit product info.
- **Actors:** Admin 1, Admin 2 (Warehouse).
- **Basic Flow:** 1. Admin enters product info -> 2. Admin uploads images (via **Multer**) -> 3. System saves.
- **Exception Flow (Delete):** Admin 2 clicks "Request Delete" -> System notifies Admin 1 -> Admin 1 clicks "Approve Delete".

UC-ADM-07: Manage Categories & Colors

- **Summary:** As an **Admin**, I want to organize products into 4 Collections and 6 Colors.
- **Actors:** Admin 1, Admin 2.

UC-ADM-08: Configure Vouchers

- **Summary:** As an **Admin**, I want to create discount codes for marketing campaigns.
- **Actors:** Admin 1, Admin 2.

GROUP 4: SECURITY

UC-ADM-09: Login to System & Change Password

- **Summary:** As an **Admin**, I want to authenticate to access the dashboard.
- **Actors:** All Admin levels.
- **Business Rule:** Admins receive **Unlimited JWT Tokens** (Ref: architecture auth-temp.js).

DETAILED SPECIFICATION (EXAMPLE: APPROVE/REJECT ORDER)

USE CASE SPECIFICATION

Use Case ID	UC-ADM-01.2
Use Case Name	Approve/Reject Order
Description	As **Admin 1**, I want to review orders submitted by Admin 3 to ensure they are valid before shipping.
Actor(s)	Admin 1 (Main Admin)
Priority	Must Have
Trigger	Admin 1 receives an email notification from Admin 3's proposal.
Pre-Condition	1. Admin 1 is logged in. 2. Order status is "Pending Review".
Post-Condition	1. Order status changes to "Approved" or "Rejected". 2. An email is sent to the Customer via **Nodemailer**.
Basic Flow	1. Admin 1 accesses the Order Management dashboard. 2. Admin 1 filters for "Pending Review" orders. 3. Admin 1 reviews customer info and inventory. 4. Admin 1 clicks "Approve". 5. System updates orders table and logs the action.

Alternative Flow	**4a. Reject Order:** 4a1. Admin 1 clicks "Reject". 4a2. Admin 1 provides a reason. 4a3. System notifies Admin 3 and Customer.
Exception Flow	**5a. Out of Sync:** Inventory is insufficient at the moment of approval -> System displays warning and stops approval.
Business Rules	BR-ADM: Only Admin Level 1 has the right to finalize the Approval/Rejection.
Non-Functional	NFR-ADM: Approval status must update in real-time for Admin 3's dashboard.

Group 1: System & Security

- Login to System: Admins must log in to access the administration dashboard.
- Change Password: Admins can change their own login password for security purposes.
- View Activity Logs: Review the history of important actions performed on the system (who did what, and when) for auditing and control.

Group 2: Customer Management

- View User List: Displays a list of all registered member accounts on the system.
- Lock/Unlock Account: An extension function from the user list, allowing Admins to lock violating accounts or unlock them.
- View Purchase History: An extension function from the user list, allowing Admins to view the detailed order history of a specific customer.

Group 3: Sales Management

- Manage Orders: View a list of all customer orders, filterable by status (e.g., new, processing, delivered, etc.).

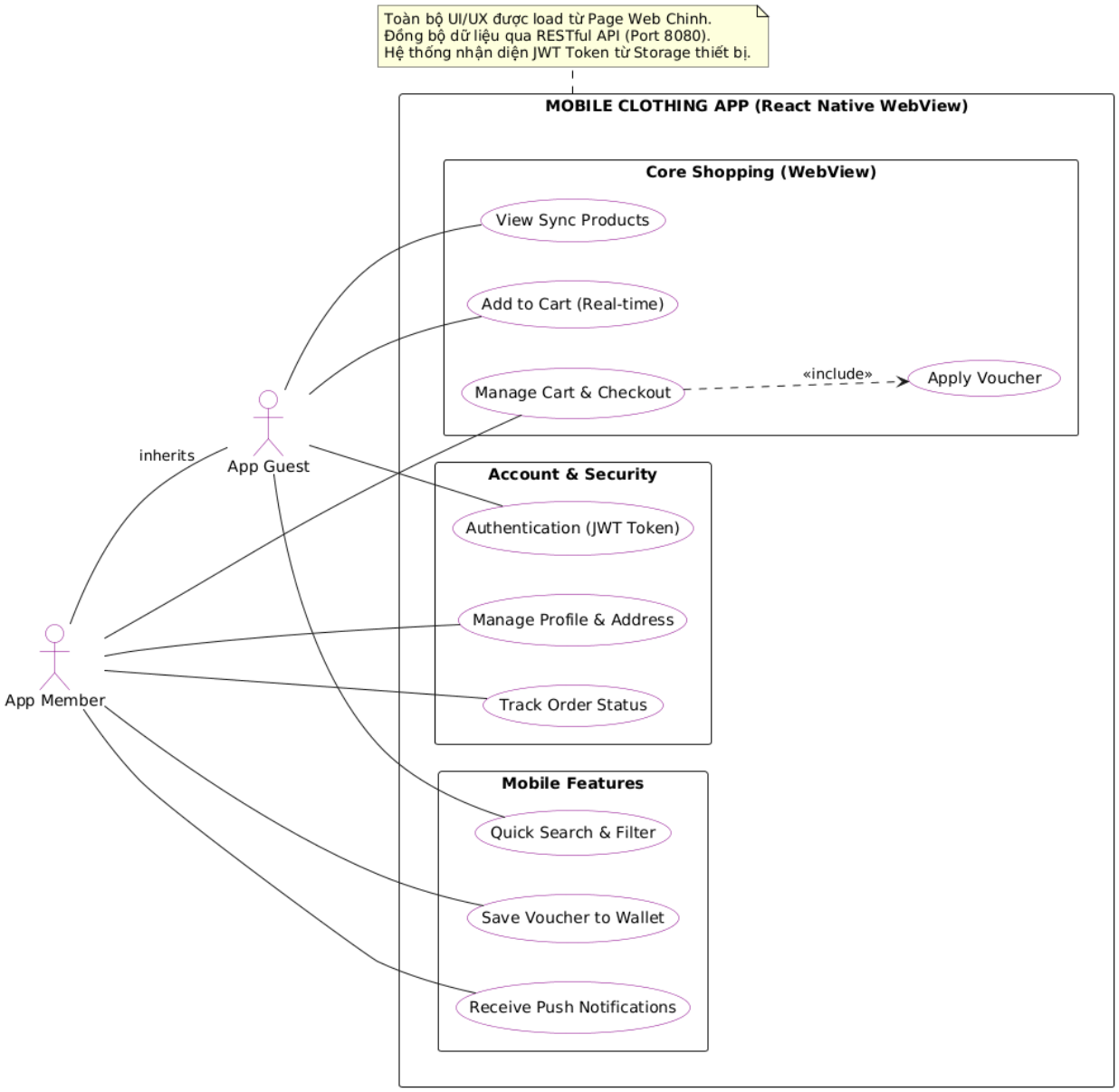
- Update Order Status: An extension function when viewing orders, allowing Admins to change the order status (e.g., from "Pending Confirmation" to "Delivering").

Group 4: Inventory & Promotions

- Manage Products: Add new products, edit information, delete or hide existing products, and manage inventory quantities.
- Manage Categories: Create, edit, or delete product categories (e.g., Men's Shirts, Women's Pants) to classify merchandise.
- Manage Sizes & Colors: Define common attributes for products, such as size charts (S, M, L) and color palettes.
- Configure Product Vouchers: Create and configure discount codes (discount amount/percentage, applicable conditions, expiration dates), which can be applied to specific products or the entire shop.

---

4. Giải pháp triển khai và Cấu trúc đồng bộ Mobile App



GROUP 1: CORE SHOPPING (WEBVIEW-BASED)

UC-MOB-01: View Sync Products

- Summary: As an App Guest, I want to browse products that are synchronized in real-time with the website.
- Actor: App Guest (Inherited by App Member).
- Technical Detail: The app loads UI/UX from the main website via React Native WebView.
- Basic Flow: 1. User opens the App -> 2. WebView points to the web URL -> 3. System fetches products via RESTful API (Port 8080) -> 4. Products display on mobile screen.

UC-MOB-02: Add to Cart (Real-time)

- Summary: As a User, I want to add items to my cart, ensuring the data is consistent across both Web and Mobile.
- Actor: App Guest.
- Post-condition: Item is added to the shared SQL Server database via API.

UC-MOB-03: Manage Cart & Checkout (Include: Apply Voucher)

- Summary: As an App Member, I want to finalize my purchase on my mobile device.
- Actor: App Member.
- Include: This Use Case mandatory includes the Apply Voucher step to calculate discounts.

GROUP 2: ACCOUNT & SECURITY

UC-MOB-04: Authentication (JWT Token)

- Summary: As an App Guest, I want to log in using my existing credentials.
- Actor: App Guest.
- Technical Detail: The system identifies and stores the JWT Token in the device's local storage (AsyncStorage) to maintain the session.

UC-MOB-05: Manage Profile & Address

- Summary: As an App Member, I want to update my shipping address for faster mobile checkout.
- Actor: App Member.

UC-MOB-06: Track Order Status

- Summary: As an App Member, I want to view the current status of my orders (Pending, Shipping, Completed).
- Actor: App Member.

GROUP 3: MOBILE FEATURES

UC-MOB-07: Quick Search & Filter

- Summary: As an App Guest, I want to use mobile-optimized search and filters.
- Actor: App Guest.

UC-MOB-08: Save Voucher to Wallet

- Summary: As an App Member, I want to save promotional vouchers into my personal "Wallet" for later use.
- Actor: App Member.

UC-MOB-09: Receive Push Notifications

- Summary: As an App Member, I want to receive instant alerts about order updates or promotions.
- Actor: App Member, Backend System.

DETAILED SPECIFICATION (EXAMPLE: AUTHENTICATION - JWT TOKEN)

USE CASE SPECIFICATION

Use Case ID	UC-MOB-04
Use Case Name	Authentication (JWT Token)
Description	As an **App Guest**, I want to log in to the Mobile App so the system recognizes me as an **App Member**.
Actor(s)	App Guest
Priority	High (Critical)
Trigger	User clicks the "Login" button within the App's account section.
Pre-Condition	1. User has an account created on the Website. 2. Device has internet access to reach the REST API.
Post-Condition	1. User is successfully authenticated. 2. **JWT Token** is saved in device storage. 3. UI updates to show Member features.
Basic Flow	1. User enters Email and Password in the WebView login form. 2. WebView sends a POST request to /api/auth-temp. 3. Backend validates credentials and returns a JWT Token. 4. App stores the Token in the device's storage. 5. User is redirected to the App's home screen.
Exception Flow	3a. Invalid Login: System displays error message; JWT is not generated. 4a. Storage Error: App fails to save Token; User remains as Guest.

Business Rules	BR-MOB: The mobile session uses the same JWT logic as the web but may have different expiration handling.
Non-Functional	NFR-MOB: Login verification must complete within 3 seconds under normal network conditions.

1. Feature Group: Mobile Experience

This group focuses on engaging users as soon as they open the app, helping them quickly search for and save offers.

- **View Featured Products:**
  - Function: Displays a list of "hot" items, new arrivals, or seasonal collections directly on the Home Screen.
  - How it works: Data is fetched from the API. The UI is designed as a grid or a horizontal carousel, allowing users to easily swipe through items.
- **Perform Quick Search:**
  - Function: Provides a product search tool right at the user's fingertips.
  - How it works: The Search Bar is always visible or pinned to the top of the page. When a user enters a keyword, the app can instantly suggest results (Auto-complete) to save typing time on mobile.
- **Save Voucher to Wallet:**
  - Function: Allows users to save discount codes seen on advertising banners into their personal "Voucher Wallet."
  - How it works: Users tap the "Save" button on a discount code. This code is then stored in the database linked to their account, meaning they don't have to memorize it for future checkout.

---

2. Feature Group: Cart & Checkout

This group manages the shopping process, emphasizing real-time data synchronization between the Website and the App.

- **View Synced Cart:**
  - Function: View the list of products currently in the shopping cart.
  - How it works: This is a crucial feature. Cart data is synced via a RESTful API. If a user adds an item to their cart on the Website (PC), they will immediately see it in their mobile app cart upon opening it, and vice versa.
- **Add Item to Cart:**
  - Function: Select products to add to the cart in preparation for purchase.
  - How it works: On the product detail page, the user selects the Size/Color and taps the "Add to Cart" button. The system updates the cart status instantly.
- **Proceed to Checkout:**
  - Function: Execute the order confirmation and payment process.
  - How it works: The checkout UI on the app is simplified (e.g., broken down into small steps: Address -> Shipping -> Payment) to fit small screens, supporting auto-fill for saved information.
- **Apply Voucher Code:**
  - Function: Use a discount code to deduct money from the order total.
  - How it works: This function is included in the checkout process. Users can select a code from their previously saved "Voucher Wallet" or enter one manually. The system automatically recalculates the total amount.

---

3. Feature Group: Personalization

This group serves registered members, helping them manage their accounts and orders.

- **Login/Register (Token-based):**
  - Function: Authenticate user identity to access member-exclusive features.
  - How it works: Utilizes Token-based Authentication (like JWT). Upon the first successful login, the app saves a secure Token. Consequently, users don't need to re-enter their password every time they open the app (unless the token expires or they actively log out), creating a seamless experience.
- **Manage Personal Info:**
  - Function: View and edit user profiles.
  - How it works: Users can update their Avatar (potentially taking a photo directly from the phone's camera), change their phone number, or manage their shipping address book.
- **Track Order Status:**
  - Function: Helps users know the whereabouts of their orders.
  - How it works: Users view a list of orders and their current status (e.g., Confirmed, Delivering, Delivered). On the Mobile App, this feature is often combined with Push Notifications to alert users the moment their order status changes.

---

5. Synchronization Architecture (English Version)

5.1. Optimized Synchronization Architecture (Backend-as-a-Service)

The system operates on a centralized data model, ensuring a seamless user experience between the Web and Mobile App environments:

- Unified Database: Both the Website and Mobile App share a single SQL Server database to ensure absolute consistency in inventory and customer information.

- Hybrid WebView Architecture: The Mobile App functions as a "Native Wrapper" developed with React Native to encapsulate the entire core Web platform. This solution leverages the Responsive Design of the ReactJS frontend, ensuring that any updates to products or promotions on the Web are reflected instantly in the App without requiring users to reinstall or update the application.

5.2. Mobile App Implementation Technologies

- React Native Core: Serves as the foundational framework to create installable applications for both iOS and Android (.apk/.ipa).
- React Native WebView: The core component that allows direct embedding of the React-based Website into the mobile application with high performance and a smooth interface.
- Token-based Authentication: Utilizes JWT (JSON Web Token) to maintain user login sessions across both Web and App platforms via local device storage.

5.3. Key Mobile Functionalities

The application focuses on optimizing the mobile experience (touch/swipe gestures) based on the ReactJS responsive framework:

- Synchronized Shopping Experience: Users can start selecting items on the Website and proceed to checkout on the Mobile App thanks to real-time cart synchronization via RESTful APIs.
- Personalized Management: Users can view order history, update personal profiles, and manage their voucher wallet directly on their mobile devices.
- Supplementary Features: Leverages native mobile capabilities for quick searches and push notifications regarding order status updates.